SHETH L.U.J & SIR M.V COLLEGE

(CYBER INFORMATION & SECURITY)

CODE AND OUTPUT OF CLI-

rsa_cli.py - C:/Users/mvluc/Desktop/t087/rsa_cli.py (3.14.6)

File Edit Format Run Options Window Help

```
import random
from math import gcd

# ---------- RSA FUNCTIONS ----------

def is_prime(n):
    if n <= 1:
        return False
    for i in range(2, int(n**0.5)+1):
        if n % i == 0:
            return False
    return True

def generate_prime():
    while True:
        p = random.randint(100,300)
        if is_prime(p):
            return p

def mod_inverse(e, phi):
    for d in range(2, phi):
        if (d*e) % phi == 1:
            return d

def generate_keys():
    p = generate_prime()
    q = generate_prime()

    while p == q:
        q = generate_prime()

    n = p*q
    phi = (p-1)*(q-1)

    e = 65537

    if gcd(e, phi) != 1:
        e = 3
        while gcd(e, phi) != 1:
            e += 2

    d = mod_inverse(e, phi)

    return (e,n),(d,n)

def encrypt(message, public_key):
```

IDLE Shell 3.14.6

File Edit Shell Debug Options Window Help

```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
================ RESTART: C:/Users/mvluc/Desktop/t087/rsa_cli.py ===============
==================================================
RSA PUBLIC KEY ENCRYPTION (CLI)
==================================================

Public Key : (65537, 43357)
Private Key: (34653, 43357)

Enter Message : SNIPER NO SNIPINGGG!

Encrypted Message
[3728, 828, 29897, 18766, 36984, 8747, 4191, 828, 40542, 4191, 3728, 828, 29897,
 18766, 29897, 828, 13176, 13176, 13176, 40649]

Decrypted Message
SNIPER NO SNIPINGGG!

----------- Security Considerations -----------
• RSA is an asymmetric encryption algorithm.
• Public key encrypts data.
• Private key decrypts data.
• Real RSA uses 2048-bit or larger keys.
• This implementation uses small prime numbers for educational purposes.
>>>
```

Ln: 26 Col: 0

27°C Cloudy Search 8:26 PM 7/19/2026

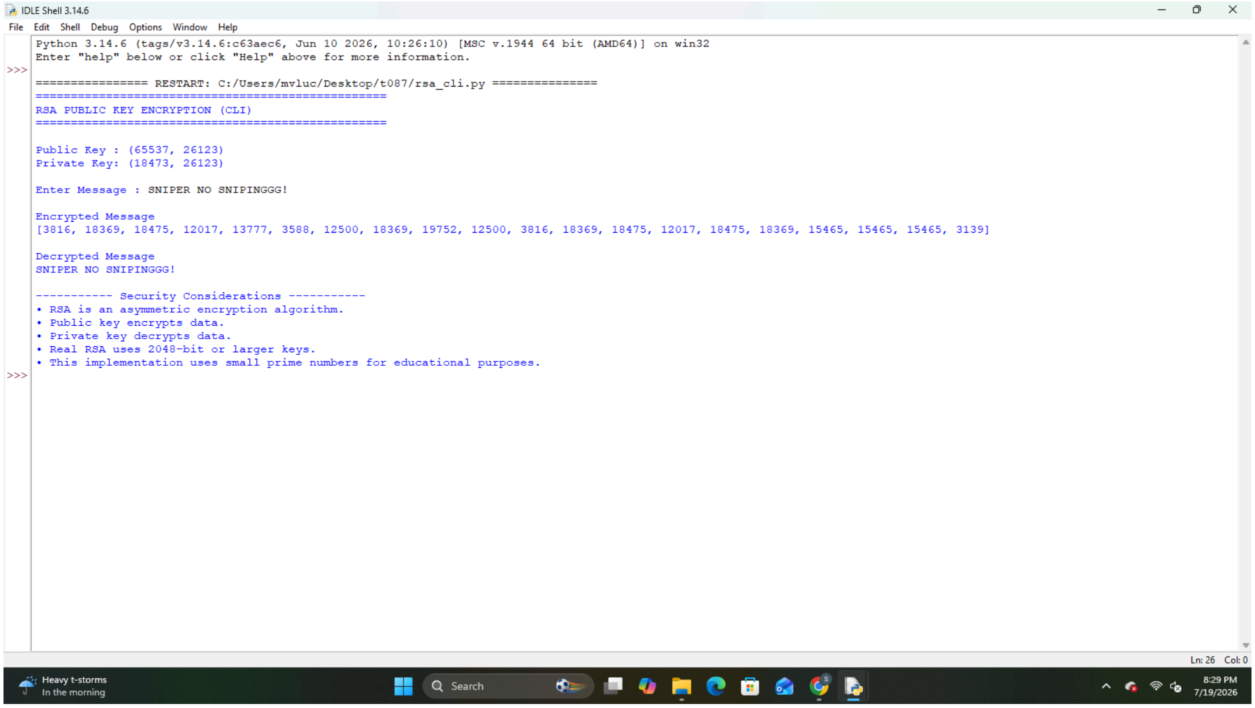
rsa_cli.py - C:/Users/mvluc/Desktop/t087/rsa_cli.py (3.14.6)

File Edit Format Run Options Window Help

```
        e = 3
        while gcd(e, phi) != 1:
            e += 2

    d = mod_inverse(e, phi)

    return (e,n),(d,n)

def encrypt(message, public_key):
    e,n = public_key
    return [pow(ord(ch),e,n) for ch in message]

def decrypt(cipher, private_key):
    d,n = private_key
    return ''.join(chr(pow(c,d,n)) for c in cipher)

# ---------- MAIN ----------

public_key, private_key = generate_keys()

print("="*50)
print("RSA PUBLIC KEY ENCRYPTION (CLI)")
print("="*50)

print("\nPublic Key :",public_key)
print("Private Key:",private_key)

message=input("\nEnter Message : ")

cipher=encrypt(message,public_key)

print("\nEncrypted Message")
print(cipher)

plain=decrypt(cipher,private_key)

print("\nDecrypted Message")
print(plain)

print("\n----------- Security Considerations -----------")
print("• RSA is an asymmetric encryption algorithm.")
print("• Public key encrypts data.")
print("• Private key decrypts data.")
print("• Real RSA uses 2048-bit or larger keys.")
print("• This implementation uses small prime numbers for educational purposes.")
```

Ln: 6 Col: 0

Heavy t-storms In the morning Search 8:28 PM 7/19/2026



```
Python 3.14.6 (tags/v3.14.6:c63aec6, Jun 10 2026, 10:26:10) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
================ RESTART: C:/Users/mvluc/Desktop/t087/rsa_cli.py ===============
==================================================
RSA PUBLIC KEY ENCRYPTION (CLI)
==================================================

Public Key : (65537, 26123)
Private Key: (18473, 26123)

Enter Message : SNIPER NO SNIPINGGG!

Encrypted Message
[3816, 18369, 18475, 12017, 13777, 3588, 12500, 18369, 19752, 12500, 3816, 18369, 18475, 12017, 18475, 18369, 15465, 15465, 15465, 3139]

Decrypted Message
SNIPER NO SNIPINGGG!

----------- Security Considerations -----------
• RSA is an asymmetric encryption algorithm.
• Public key encrypts data.
• Private key decrypts data.
• Real RSA uses 2048-bit or larger keys.
• This implementation uses small prime numbers for educational purposes.
>>>
```

CODE AND OUTPUT OF GUI-

SHETH L.U.J & SIR M.V COLLEGE

(CYBER INFORMATION & SECURITY)

rsa_gui.py - C:/Users/mvluc/Desktop/t087/rsa_gui.py (3.14.6)

File Edit Format Run Options Window Help

```python
import random
from math import gcd
import tkinter as tk
from tkinter import messagebox

# ---------- RSA FUNCTIONS ----------

def is_prime(n):
    if n <= 1:
        return False
    for i in range(2,int(n**0.5)+1):
        if n%i==0:
            return False
    return True

def generate_prime():
    while True:
        p=random.randint(100,300)
        if is_prime(p):
            return p

def mod_inverse(e,phi):
    for d in range(2,phi):
        if (d*e)%phi==1:
            return d

def generate_keys():

    p=generate_prime()
    q=generate_prime()

    while p==q:
        q=generate_prime()

    n=p*q
    phi=(p-1)*(q-1)

    e=65537

    if gcd(e,phi)!=1:
        e=3
        while gcd(e,phi)!=1:
            e+=2

    d=mod_inverse(e,phi)
```

Ln: 132 Col: 0

Rainy days ahead 27°C Search 8:32 PM 7/19/2026

rsa_gui.py - C:/Users/mvluc/Desktop/t087/rsa_gui.py (3.14.6)

File Edit Format Run Options Window Help

```python
        e=3
        while gcd(e,phi)!=1:
            e+=2

    d=mod_inverse(e,phi)

    return (e,n),(d,n)

def encrypt(message,key):
    e,n=key
    return [pow(ord(ch),e,n) for ch in message]

def decrypt(cipher,key):
    d,n=key
    return ''.join(chr(pow(c,d,n)) for c in cipher)

public_key,private_key=generate_keys()

def process():

    msg=entry.get()

    if msg=="":
        messagebox.showerror("Error","Enter a message")
        return

    cipher=encrypt(msg,public_key)
    plain=decrypt(cipher,private_key)

    txt_public.delete("1.0",tk.END)
    txt_public.insert(tk.END,str(public_key))

    txt_private.delete("1.0",tk.END)
    txt_private.insert(tk.END,str(private_key))

    txt_cipher.delete("1.0",tk.END)
    txt_cipher.insert(tk.END,str(cipher))

    txt_plain.delete("1.0",tk.END)
    txt_plain.insert(tk.END,plain)

window=tk.Tk()

window.title("RSA Encryption and Decryption")
window.geometry("720x650")
```

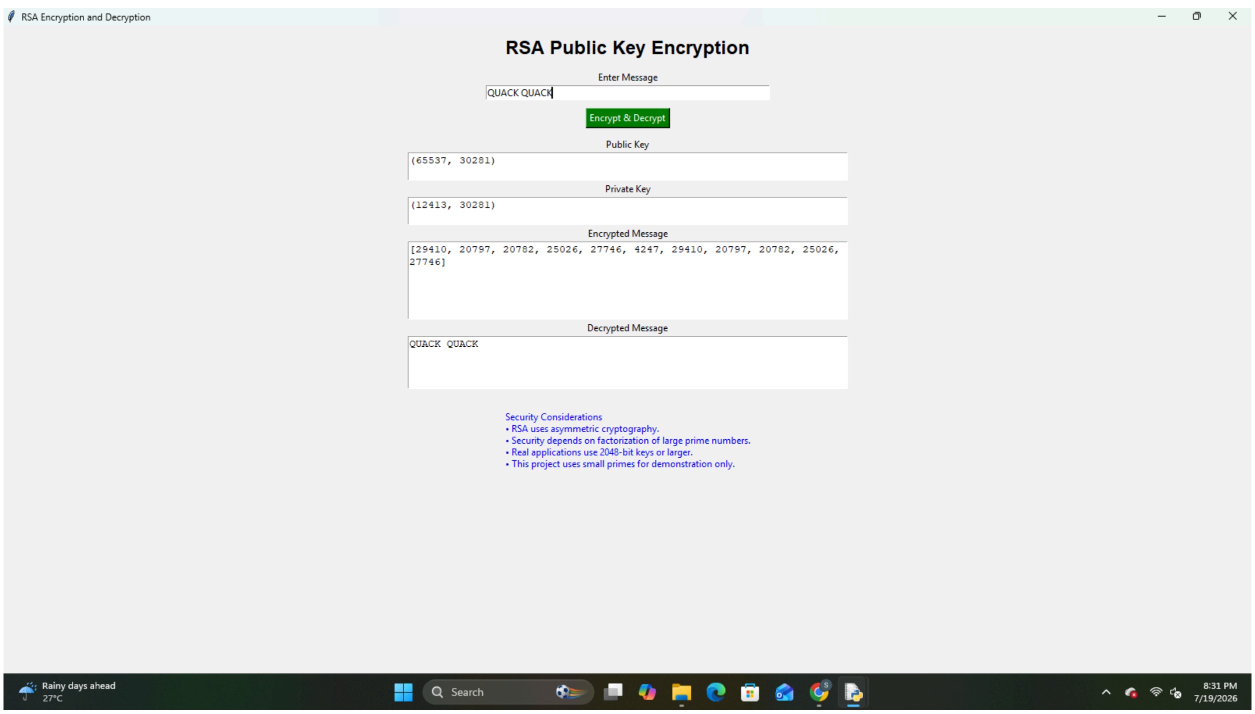
Ln: 132 Col: 0

Rainy days ahead 27°C Search 8:32 PM 7/19/2026

SHETH L.U.J & SIR M.V COLLEGE

(CYBER INFORMATION & SECURITY)

```python
tk.Label(window,
text="RSA Public Key Encryption",
font=("Arial",18,"bold")).pack(pady=10)

tk.Label(window,text="Enter Message").pack()

entry=tk.Entry(window,width=60)
entry.pack()

tk.Button(window,
text="Encrypt & Decrypt",
bg="green",
fg="white",
command=process).pack(pady=10)

tk.Label(window,text="Public Key").pack()

txt_public=tk.Text(window,height=2,width=70)
txt_public.pack()

tk.Label(window,text="Private Key").pack()

txt_private=tk.Text(window,height=2,width=70)
txt_private.pack()

tk.Label(window,text="Encrypted Message").pack()

txt_cipher=tk.Text(window,height=6,width=70)
txt_cipher.pack()

tk.Label(window,text="Decrypted Message").pack()

txt_plain=tk.Text(window,height=4,width=70)
txt_plain.pack()

tk.Label(window,
text="\nSecurity Considerations\n"
"• RSA uses asymmetric cryptography.\n"
"• Security depends on factorization of large prime numbers.\n"
"• Real applications use 2048-bit keys or larger.\n"
"• This project uses small primes for demonstration only.",
fg="blue",
justify="left").pack(pady=10)

window.mainloop()
```



SHETH L.U.J & SIR M.V COLLEGE
(CYBER INFORMATION & SECURITY)